Algorithms of Machine Learning Experiments

Derived from the notebook *Veruthee.ipynb*

Contents

1. Regression (Simple Linear, Multiple Linear, Polynomial)
2. K-Nearest Neighbors (KNN) Classification
3. Naive Bayes Classifier (NBC)
4. Hierarchical Agglomerative Clustering (HAC)
5. K-Means Clustering (KMC)

1. Regression

Dataset: Advertising.csv (features TV, Radio, Newspaper; target Sales). Metrics: Mean Squared Error (MSE) and R^2 score. Train/test split = 80/20, random_state = 42.

1a. Simple Linear Regression

Algorithm: Simple Linear Regression

Input: Advertising.csv

Output: MSE, R2, regression line plot

1. Load the dataset into a DataFrame.
2. Select feature X = TV and target y = Sales.
3. Split into training and test sets (test_size = 0.2, random_state = 42).
4. Fit a Linear Regression model on the training data: y = b0 + b1*TV.
5. Predict Sales for the test set.
6. Compute MSE and R2 between actual and predicted test values.
7. Plot the scatter of TV vs Sales and overlay the fitted line (red).

1b. Multiple Linear Regression

Algorithm: Multiple Linear Regression

Input: Advertising.csv

Output: MSE, R2, actual vs predicted plot

1. Select features X = [TV, Radio, Newspaper] and target y = Sales.
2. Split into training and test sets (80/20, random_state = 42).
3. Fit a Linear Regression model: y = b0 + b1*TV + b2*Radio + b3*Newspaper.
4. Predict Sales for the test set.
5. Compute MSE and R2 on the test set.
6. Plot a scatter of actual Sales vs predicted Sales.

1c. Polynomial Regression (degree 2)

Algorithm: Polynomial Regression

Input: Advertising.csv

Output: MSE, R2, polynomial curve plot

1. Select feature X = TV and target y = Sales.
2. Transform X using PolynomialFeatures(degree = 2) to obtain [1, TV, TV^2].
3. Split the transformed data into training and test sets (80/20, random_state = 42).
4. Fit a Linear Regression model on the polynomial features: y = b0 + b1*TV + b2*TV^2.
5. Predict Sales for the test set.
6. Compute MSE and R2 on the test set.
7. Sort TV values, transform them with the same polynomial transformer, and plot the fitted curve over the scatter of TV vs Sales.

2. K-Nearest Neighbors (KNN) Classification

Dataset: Iris (150 samples, 4 features, 3 classes). K = 5.

Algorithm: KNN Classifier

Input: Iris dataset, K = 5

Output: Accuracy, macro Precision, macro Recall

1. Load the Iris dataset; obtain feature matrix X and labels y.
2. Standardize features using StandardScaler (zero mean, unit variance).
3. Split into training and test sets (test_size = 0.2, random_state = 42).
4. Train: store the training samples (lazy learner) with n_neighbors = 5.
5. For each test sample: compute the Euclidean distance to all training samples, select the K = 5 nearest, and assign the majority class among them.
6. Collect the predictions y_pred.
7. Evaluate: Accuracy, Precision (macro average) and Recall (macro average) against y_test.

3. Naive Bayes Classifier (NBC)

Dataset: Iris. Model: Gaussian Naive Bayes.

Algorithm: Gaussian Naive Bayes

Input: Iris dataset

Output: Accuracy

1. Load the Iris dataset; obtain X and y.
2. Standardize features using StandardScaler.
3. Split into training and test sets (test_size = 0.2, random_state = 42).
4. Training: for each class c compute the prior P(c) and, for each feature j, the mean and variance of feature j within class c.
5. Prediction: for each test sample x, compute for every class P(c|x) proportional to P(c) * product over j of N(x_j; mean_cj, var_cj), where N is the Gaussian density.
6. Assign the class with the highest posterior probability.
7. Evaluate Accuracy of y_pred against y_test.

4. Hierarchical Agglomerative Clustering (HAC)

Dataset: synthetic blobs (150 samples, 3 centers, random_state = 42). Linkage: Ward.

Algorithm: Agglomerative Clustering (Ward linkage)

Input: make_blobs(n_samples = 150, centers = 3), n_clusters = 3

Output: Dendrogram, cluster labels, cluster plot

1. Generate the synthetic dataset X using make_blobs and ignore the true labels.
2. Standardize X using StandardScaler.
3. Initialize: treat each of the 150 samples as its own cluster.
4. Compute the linkage matrix with Ward's method: repeatedly merge the pair of clusters whose merge gives the smallest increase in total within-cluster variance, until a single cluster remains.
5. Plot the dendrogram from the linkage matrix (x: samples, y: merge distance).
6. Cut the tree to obtain 3 clusters using AgglomerativeClustering(n_clusters = 3, linkage = 'ward') and get labels via fit_predict.
7. Visualize the clusters with a scatter plot of Feature 1 vs Feature 2 coloured by cluster label.

5. K-Means Clustering (KMC)

Dataset: synthetic blobs (150 samples, 3 centers, random_state = 42).

Algorithm: K-Means with Elbow Method

Input: make_blobs(n_samples = 150, centers = 3), K = 3, K range 1 to 9

Output: Cluster labels, centroids, elbow plot, cluster plot

1. Generate the synthetic dataset X using make_blobs and standardize it with StandardScaler.
2. Initialize K = 3 centroids (k-means++ initialization, random_state = 42).
3. Assignment step: assign every point to its nearest centroid (Euclidean distance).
4. Update step: recompute each centroid as the mean of the points assigned to it.
5. Repeat steps 3 and 4 until the assignments stop changing (convergence) or the maximum iterations are reached.
6. Elbow Method: for each k = 1, ..., 9 fit K-Means and record the inertia (sum of squared distances to the nearest centroid); plot inertia vs k and pick the 'elbow' as the optimal K.
7. Display the cluster assignments and centroids.
8. Visualize the clusters in a scatter plot coloured by label, with centroids marked as red X.

Libraries used: pandas, numpy, matplotlib, scikit-learn, scipy.